JAVA PROJECT REPORT ON

SMART TO-DO TASK MANAGER

Submitted by

SHAMITHA J

Java Programming

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

KG College of Arts and Science

Academic Year: 2025 – 2026

APRIL 2026

Semester: I

DECLARATION

I hereby declare that the project entitled “Smart To-Do Task Manager” submitted in partial fulfillment of the requirements for the course Java Programming is a record of my work carried out for academic purposes.

This project is implemented as a Java desktop application using Swing components and event handling. The application allows a user to add tasks, assign priorities, mark selected tasks as completed, delete tasks, and sort tasks by priority when a task is added.

I have acknowledged the Java documentation sources used for background understanding in the References section.

April 2026

Coimbatore

_____

SHAMITHA J

CERTIFICATE

This is to certify that the project entitled “Smart To-Do Task Manager” is submitted by **SHAMITHA J**, B.Sc. Information Technology, KG College of Arts and Science, in partial fulfillment of the requirements for the course Java Programming.

This project report describes a Java Swing application for maintaining a task list, assigning High, Medium, or Low priority, marking tasks as completed, and deleting selected tasks.

Place: Coimbatore

Date: 09.04.2026

Submitted for the Viva-Voce Examination held on

09.04.2026

Guide: (Technical Trainer, KGCAS)

_____

Head of the Department

_____

TABLE OF CONTENTS

S. No.	Topics
1	Introduction
1.1	Objective
1.2	Overview
1.3	Features
2	Program Analysis
2.1	Problem Definition
2.2	Project Overview
2.3	Scopes and Limitations
3	System Design
3.1	Modules
4	Methodology
4.1	Problem Analysis
4.2	System Design
4.3	Program Development
4.4	Testing and Validation
4.5	Result and Documentation
4.6	Algorithm
4.7	Flowchart
5	Source Code
6	Input and Output Screens
7	Discussion and Conclusion
7.1	Discussion
7.2	Conclusion
8	References

ABSTRACT

The Smart To-Do Task Manager is a Java-based desktop application designed to help users organize daily tasks in a simple graphical interface. The user can enter a task description, select a priority level, and add it to a list. The available priorities are High, Medium, and Low.

The application also allows users to mark a selected task as completed and delete a selected task. When a task is added, the program sorts the list according to priority so that High priority tasks are placed before Medium and Low priority tasks. Validation messages are displayed when the task field is empty or when an action requiring a selection is performed without selecting a task.

The system is developed using Java Swing components, including JFrame, JLabel, JTextField, JComboBox, JButton, JList, DefaultListModel, and JScrollPane. ActionListener handles button events, while ArrayList sorting is used to organize task entries. This project demonstrates GUI development, event-driven programming, conditional statements, collection handling, and basic sorting. Tasks are kept in memory and are not saved after the application closes.

1. INTRODUCTION

Task management is important for organizing academic assignments, personal reminders, and everyday responsibilities. When several tasks need attention, a list with priority levels can help users identify which tasks should be handled first.

The Smart To-Do Task Manager is a desktop application developed using Java Swing. It provides a window where users can enter task descriptions, select priorities, and manage tasks with buttons. The project demonstrates the practical use of Java programming concepts in a small graphical application.

1.1 Objective

- To develop a simple Java desktop application for managing tasks.
- To accept task descriptions through a text field.
- To provide High, Medium, and Low priority options.
- To sort tasks by priority when a new task is added.
- To allow users to mark selected tasks as completed or delete them.
- To demonstrate Swing components and event-driven programming.

1.2 Overview

When the program starts, a window titled “Smart To-Do Task Manager” appears. The user enters a task in the text field and chooses a priority from the drop-down box. Clicking Add Task adds the task to the list, clears the text field, sorts the list, and displays a confirmation message. To complete or delete a task, the user selects it from the list and clicks the corresponding button.

1.3 Features

- Graphical interface developed with Java Swing.
- Task entry using a text field.
- Priority selection through a drop-down list.
- Automatic priority sorting when adding tasks.
- Completion marking using the [Completed] prefix.
- Deletion of a selected task.
- Validation messages for empty input and missing selection.
- Scrollable list for displaying tasks.

2. PROGRAM ANALYSIS

Program analysis identifies the problem addressed by the application, its behaviour, and its limitations. The supplied program focuses on basic task-list management within a single desktop window.

2.1 Problem Definition

Keeping track of tasks manually can make it difficult to identify urgent work and maintain an organized list. A simple digital task manager can centralize task descriptions and indicate their importance. This project addresses that need through a small interface with priority labels and basic task operations.

2.2 Project Overview

The application is implemented in the class SmartTodoGUI, which extends JFrame and implements ActionListener. The constructor creates the window and its controls. The actionPerformed() method responds to button clicks and updates the task list. The sortTasks() method copies task strings into an ArrayList, sorts them by the priority value returned by getPriority(), and adds them back to the list model.

Priority values are ranked as High = 1, Medium = 2, and Low = 3. Therefore, tasks are sorted in that order whenever Add Task is used. Completing or deleting a task does not trigger sorting in the current implementation.

2.3 Scopes and Limitations

Scope

- Adding tasks with a chosen priority.
- Displaying tasks in a scrollable list.
- Sorting tasks by priority after task addition.
- Marking selected tasks as completed.
- Deleting selected tasks.

Limitations

- Tasks are stored only in memory and are lost when the program closes.
- The application has no database or file storage.
- Task editing, due dates, reminders, search, and filtering are not implemented.
- The interface uses fixed positions with setBounds(), which limits flexible resizing.
- Priority sorting is called after adding a task, not after completion or deletion.
- Completed tasks remain in the list and are identified by a prefix.

3. SYSTEM DESIGN

The system follows an event-driven design. The user interacts with the Swing window, and each button click is handled by actionPerformed(). The method checks the source of the event and carries out the required task operation.

3.1 Modules

1. User Interface Module

Creates the main JFrame window and displays labels, the task text field, priority selector, buttons, and scrollable list.

2. Task Input Module

Reads the task description from the text field and obtains the selected priority from the combo box.

3. Add Task Module

Checks whether the task text is empty. If valid, it adds the task and priority to the list, clears the field, sorts the tasks, and displays a success message.

4. Priority Sorting Module

Uses an ArrayList and the getPriority() method to place High priority tasks before Medium and Low priority tasks.

5. Completion Module

Checks the selected list item and adds the [Completed] prefix if the task is not already completed.

6. Delete Module

Removes the selected task from the DefaultListModel.

7. Validation and Message Module

Displays messages for empty input, successful actions, missing selections, and tasks that are already completed.

4. METHODOLOGY

The development methodology consists of identifying the task-management requirements, designing the interface, implementing the Java code, and checking expected behaviour with manual test cases.

4.1 Problem Analysis

The main requirements are task entry, priority selection, task display, task completion, deletion, and priority ordering. The task list uses DefaultListModel so that the displayed items can be updated when the user performs an action.

4.2 System Design

A JFrame contains the user interface. JTextField accepts the task description, JComboBox supplies the priority choices, and JButton controls the actions. JList displays the task strings inside a JScrollPane. DefaultListModel stores the strings shown by the list.

4.3 Program Development

- Import Swing, AWT, and event-handling packages.
- Create the SmartTodoGUI class extending JFrame and implementing ActionListener.
- Set the window title, size, close operation, layout, and background.
- Create labels, text field, priority selector, buttons, and scroll pane.
- Attach action listeners to the Add Task, Complete, and Delete buttons.
- Implement the button actions in actionPerformed().
- Implement sortTasks() and getPriority() for priority-based ordering.
- Launch the GUI using SwingUtilities.invokeLater().

4.4 Testing and Validation

The following test cases describe expected behaviour based on the supplied code. They are suggested manual checks and do not imply that an automated test suite is included.

Test Case	Action / Input	Expected Result
1	Enter a task and select High; click Add Task.	Task is added and the list is sorted.
2	Leave the task field empty and click Add Task.	“Please enter a task!” message appears.
3	Add High, Medium, and Low priority tasks.	List is sorted High, Medium, Low after additions.
4	Select a task and click Complete.	The task receives the [Completed] prefix.
5	Complete the same task again.	“Task is already completed!” message appears.
6	Click Complete without selecting a task.	“Please select a task!” message appears.
7	Select a task and click Delete.	Selected task is removed.
8	Click Delete without selecting a task.	“Please select a task!” message appears.

4.5 RESULT AND DOCUMENTATION

The program is designed to display a task list with priority information and provide confirmation or validation messages for user actions. When a task is added, the displayed list is sorted by priority. Completion and deletion update the list immediately.

The documentation records the purpose, design, modules, algorithm, expected test cases, and limitations of the supplied source code. Screenshots should be captured by running the application if they are required for submission.

4.6 Algorithm

- Start the application.
- Create and display the Smart To-Do Task Manager window.
- Wait for the user to click a button.
- If Add Task is clicked, read the task description and selected priority.
- If the task description is empty, display a validation message; otherwise add the task, clear the field, sort the list, and show confirmation.
- If Complete is clicked, check whether a task is selected. If none is selected, show a message. Otherwise, add [Completed] unless it is already present.
- If Delete is clicked, check whether a task is selected. If none is selected, show a message; otherwise remove the selected task.
- Continue waiting for actions until the window is closed.

4.7 FLOWCHART

START
↓
Display task manager window
↓
Wait for button action
↓
Identify selected button
Add Task: validate text → add task → sort list
Complete: check selection → mark completed
Delete: check selection → remove task
↓
Display message and update list
↓
Wait for next action / END when closed

5. SOURCE CODE

The main class is SmartTodoGUI. It extends JFrame to create the application window and implements ActionListener to process button events. The code is organized around the constructor, actionPerformed(), sortTasks(), getPriority(), and main().

Important components

- JFrame: creates the main window.
- JLabel: displays the title and field labels.
- JTextField: accepts the task description.
- JComboBox: provides High, Medium, and Low priority options.
- JButton: triggers add, complete, and delete operations.
- DefaultListModel and JList: store and display task strings.
- JScrollPane: makes the task list scrollable.
- ActionListener: connects button clicks to application logic.

Task addition logic

```
String task = taskField.getText().trim();
String priority = (String) priorityBox.getSelectedItem();

if (task.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Please enter a task!");
} else {
    taskList.addElement(task + " - Priority: " + priority);
    taskField.setText("");
    sortTasks();
    JOptionPane.showMessageDialog(this, "Task added successfully!");
}
```

Priority ranking logic

```
int getPriority(String task) {
    if (task.contains("Priority: High")) return 1;
    if (task.contains("Priority: Medium")) return 2;
    return 3;
}
```

The complete and delete actions operate on the selected list index. The main() method starts the interface with SwingUtilities.invokeLater(), which schedules GUI creation on the Swing event-dispatch thread.

6. INPUT AND OUTPUT SCREENS

The input and output of the application are handled through the graphical interface.

Input

- Task description typed into the text field.
- Priority selected from High, Medium, or Low.
- Task selected from the list for completion or deletion.

Output

- Task shown with its priority label.
- List sorted by priority after adding a task.
- Completed task shown with a [Completed] prefix.
- Selected task removed after deletion.
- Dialog messages for success and validation.

Example 1: Adding a task

Input: Task = “Submit assignment”; Priority = High.
Expected output: “Submit assignment - Priority: High” and a “Task added successfully!” dialog.

Example 2: Priority sorting

Input: Add tasks with Low, High, and Medium priorities.
Expected order after the additions: High priority, Medium priority, Low priority.

Example 3: Empty task

Input: Leave the task field blank and click Add Task.
Expected output: “Please enter a task!”

Example 4: Completing a task

Action: Select a task and click Complete.
Expected output: The task is prefixed with “[Completed]” and a confirmation dialog appears.

Screenshot note: No screenshots of a running application were supplied with the code. Add actual screenshots captured from the application if required by the department.

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Smart To-Do Task Manager demonstrates how Java Swing can be used to create an interactive desktop application. It combines graphical components, button event handling, input validation, list-model updates, and priority sorting. The priority levels help users distinguish important tasks, while the completion prefix provides a simple status indicator.

The current implementation is a basic prototype. It does not store tasks permanently and does not include task editing, due dates, reminders, or multiple users. Sorting is triggered when a task is added; completing or deleting a task does not trigger a new sort.

7.2 Conclusion

The project meets its basic objective of providing a graphical task list with priority selection and common task operations. It demonstrates practical beginner-level Java concepts such as Swing components, ActionListener, conditional logic, ArrayList sorting, and DefaultListModel updates. The project can be extended into a more complete task-management system.

Future Enhancements

- Save tasks to a file or database so they remain available after closing the program.
- Add due dates, reminders, categories, and search.
- Allow users to edit tasks and change their priority.
- Show completed and pending tasks with different visual styles.
- Re-sort the list after task status or priority changes.
- Use layout managers to improve resizing and responsiveness.

8. REFERENCES

The following sources provide background information about Java, Swing components, and event handling used in the project.

1. Oracle, Java Documentation — https://docs.oracle.com/en/java/

2. Oracle, Creating a GUI With Swing — https://docs.oracle.com/javase/tutorial/uiswing/

3. Oracle, How to Write an Action Listener — https://docs.oracle.com/javase/tutorial/uiswing/events/actionlistener.html

4. Source code supplied for this report: SmartTodoGUI.java — Smart To-Do Task Manager.

END OF REPORT